

Department of Computer Science
Faculty of Engineering, Built Environment & IT
University of Pretoria

COS 301: Software Engineering

Software Architecture Specification

Capstone Project 2026 - Demo 3

Team Name: OmniTech

September 3, 2026

Contents

1	Introduction	3
2	Project Owner	3
3	Architectural Requirements	4
3.1	Overall Software Architecture	4
3.2	Primary Architectural style	4
3.3	Communication Patterns	4
3.3.1	Request-Response Model	4
3.3.2	Push-Pull Model	4
3.3.3	Event-Driven Messaging	5
3.4	Communication Protocols and Standards	5
3.5	Architectural Patterns Per Subsystem	5
3.5.1	Backend API	5
3.5.2	Android Application	6
3.5.3	Web Dashboard	6
3.6	Design Patterns	7
3.6.1	Observer Pattern	7
3.6.2	Strategy Pattern	7
3.7	Constraints	7
3.7.1	Hardware Constraints	7
3.7.2	Platform Constraints	7
3.7.3	Real-Time Processing Constraints	7
3.7.4	Privacy and Legal Constraints	8
3.7.5	Technology Constraints	8
3.8	High-Level Architecture Diagram	9
3.9	Quality Requirements to Architectural Decisions	9
3.10	NFR Traceability Matrix	10
4	Service Contract	11
5	Technology Requirements	11
5.1	TR1 - Mobile Application Runtime:	11
5.2	TR2 - Backend Runtime:	11
5.3	TR3 - Database:	11
5.4	TR4 - Containerization:	12
5.5	TR5 - Cloud Infrastructure:	12
5.6	TR6 - Frontend Runtime:	12
5.7	TR7 - Package Management:	12

5.8	TR8 - Version Control:	12
5.9	TR9 - Android Development Environment:	12
5.10	TR10 - External APIs and Services:	12
5.11	TR11 - Authentication:	12
5.12	TR12 - CI/CD Pipeline:	12
5.13	Deployment Diagram	13
5.14	CI pipeline flow diagram	14
5.15	CD pipeline flow diagram	15

1 Introduction

Driving Tracker is an intelligent Android-based driving companion developed by OmniTech. The system aims to improve road safety, fuel efficiency, and vehicle maintenance by combining mobile sensors, GPS tracking, and vehicle diagnostics into a single integrated platform. Unlike existing solutions that focus only on navigation or diagnostics, Driving Tracker provides drivers with real-time feedback on driving behavior and vehicle health.

This document explores the software requirements and design specifications of the Driving Tracker application, as well as the architectural and technology requirements.

2 Project Owner

The project owner and client of the Driving Tracker is Gendac, and our industry mentors are Kevin Cowdrey, Sthabiso Jaffar, and Brandon Craytor.

3 Architectural Requirements

This section defines the architectural requirements for the Driving Tracker system. These requirements govern the structural decisions, quality attributes, design patterns, and constraints that guide development across the application.

3.1 Overall Software Architecture

This section discusses the high-level architecture of the Driving Tracker system across three levels of granularity, covering the primary architectural style, communication patterns between subsystems, and the architectural patterns applied within each subsystem.

3.2 Primary Architectural style

The Driving Tracker system adopts a Client-Server architecture as its primary architectural style. The system consists of two distinct client types: the Android mobile application, the web-based fleet management dashboard (and indirectly, OBD-II Bluetooth adapters as peripheral data sources). These clients interact with a centralised backend server responsible for data persistence, business logic, leaderboard computation, fleet coordination, and notification dispatch. This architecture is appropriate for Driving Tracker because the leaderboard, gamification, fleet management, and trusted contact features inherently require a shared server that multiple clients can communicate with.

3.3 Communication Patterns

Communication patterns used between subsystems:

3.3.1 Request-Response Model

The primary communication pattern between clients and the backend is the request-response model over HTTP. The Android application and the web dashboard send HTTP requests to the REST API, which processes the request and returns a JSON response. This pattern governs all standard operations including authentication, trip submission, leaderboard retrieval, vehicle management, and trusted contact alerts.

3.3.2 Push-Pull Model

A push-pull model is used for data synchronization and alerting. The Android application buffers trip and sensor data locally and pushes it to the backend in the background when connectivity is restored. Trusted contact alerts and safety notifications are pushed from the backend to the Android client via Firebase Cloud Messaging. The web dashboard pulls fleet data from the backend on demand and on a configurable polling interval.

3.3.3 Event-Driven Messaging

A event-driven messaging pattern is used to process sensor and OBD data within the Android application. Raw sensor readings are emitted as events through Kotlin Flow streams. Downstream consumers/modules, including the harsh event detector, and UI layer, subscribe to these streams and react to readings independently. This decouples data production from data consumption and allows the sampling rate to differ from the processing and rendering rate.

3.4 Communication Protocols and Standards

The application-level communication protocols and standards applied across the system are as follows:

- REST over HTTPS is used for all client-to-server API communication. REST was selected over SOAP, GraphQL, and RPC approaches because it is stateless, widely supported across Android and web clients, and sufficient for the CRUD-heavy operations of this system. GraphQL was considered but ruled out given the system does not require dynamic field selection at the client level.
- HTTP/1.1 is the underlying transport protocol. TLS 1.2 or higher is required for all connections, enforced at the server and network layer.
- Bluetooth Classic is used for OBD-II communication between the Android device and the ELM327 adapter. This is more so a hardware constraint imposed by ELM327 adapters.
- JSON is the data interchange format for all REST API responses and requests.

3.5 Architectural Patterns Per Subsystem

The following architectural patterns are applied within each subsystem, with multiple combined where appropriate:

3.5.1 Backend API

- Layered (n-tier) Architecture: The backend is structured into three distinct layers: the routing layer, the controller layer, and the service layer. Each layer has a single responsibility and communicates only with the layer directly below it.
- Service-Oriented Architecture (SOA): Backend functionality is decomposed into independently operable services including authentication, trip management, gamification, fleet management, notifications, and user management. Each service exposes a well-defined interface.

3.5.2 Android Application

- **Modular Architecture:** The Android application is divided into independently developed and tested feature modules covering sensors, OBD communication, trip tracking, gamification, notifications, and data synchronization.
- **Offline-First Architecture:** The application treats the local Room database as the primary data store. All writes are committed locally before being synchronized to the backend, ensuring uninterrupted trip recording in low-connectivity.
- **Event-Driven Architecture:** The sensor pipeline uses reactive event streams implemented with Kotlin Flow, allowing multiple subsystems to consume sensor data independently without tight coupling.

3.5.3 Web Dashboard

- **Component-Based Architecture:** follows The React and Next.js web dashboard a component-based UI architecture, decomposing screens into reusable, independently testable UI components with centralized state management.

3.6 Design Patterns

3.6.1 Observer Pattern

Sensor data streams on Android are modelled as observable flows using Kotlin Flow. UI components and scoring logic subscribe to these flows and react to new readings. This pattern decouples the sensor sampling rate from the UI refresh rate and allows multiple consumers to receive the same sensor events independently.

3.6.2 Strategy Pattern

Driver scoring and harsh event detection could use configurable strategies that encapsulate the scoring algorithm. Different vehicle types or driving profiles may use different scoring strategies without altering the core trip recording logic. This also provides an easier path to introduce ML based scoring while alongside rule-based scoring.

3.7 Constraints

3.7.1 Hardware Constraints

- OBD-II diagnostics require a physical ELM327-compatible Bluetooth adapter connected to the vehicle. Vehicles made before 1996 may not have an OBD-II port and will not support this feature.
- Sensor availability varies between Android devices. The system must implement adaptive thresholds and degrade gracefully on devices that do not have specific sensors.
- Bluetooth Classic is required for ELM327 communication. Although most devices do have Bluetooth support, those without cannot use OBD features.

3.7.2 Platform Constraints

- Background location access requires permission which needs to be explicitly granted by the user on Android 10 and above.
- Background execution is subject to Android battery optimization restrictions. Ideally, the system should use a foreground service with a persistent notification during active trip recording to prevent the system from terminating the process.

3.7.3 Real-Time Processing Constraints

- Road noise, vibration, and environmental interference could produce false positives in harsh event detection. Configurable sensitivity levels, as well as other techniques like smoothing, should be utilised to reduce false alert rates.
- Sensor and OBD data processing must not block the main thread, to avoid delaying UI

or navigation. Processing must occur on background threads or coroutines.

3.7.4 Privacy and Legal Constraints

- Location tracking must not begin until the user has granted location permissions and provided consent through the in-app consent flow.
- Live trip sharing with trusted contacts must be managed per trip. Contacts must not be able to view location data without the user actively sharing their location.

3.7.5 Technology Constraints

- Backend infrastructure must be deployable to Microsoft Azure.
- Background execution is subject to Android battery optimization restrictions. Ideally, the system should use a foreground service with a persistent notification during active trip recording to prevent the system from terminating the process.

3.8 High-Level Architecture Diagram

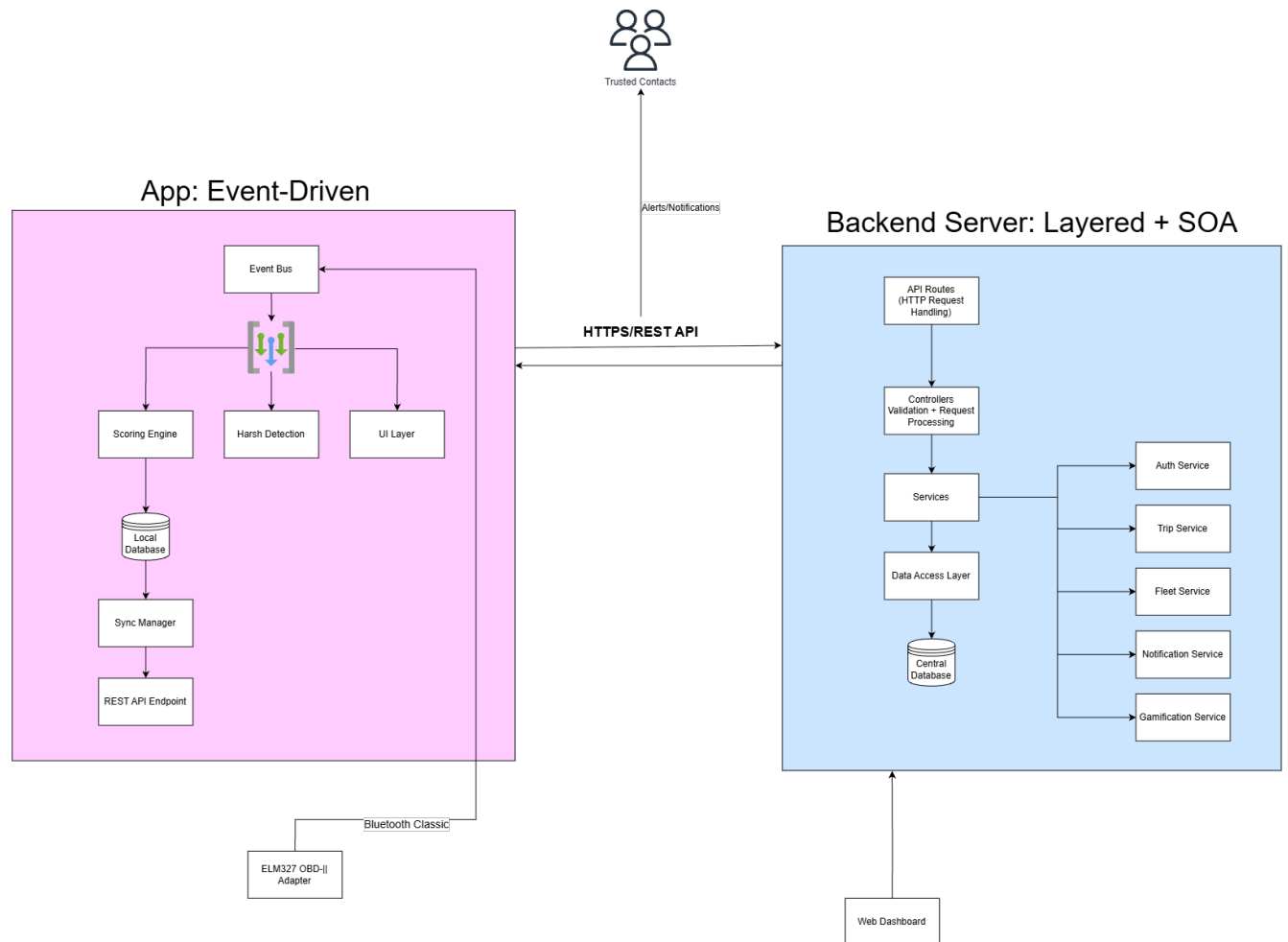


Figure 1: Architectural Diagram

3.9 Quality Requirements to Architectural Decisions

ID	Quality Requirement	Architectural Decision
QR-01	UI should respond ≤ 200 ms	Singleton obd manager and sensor fusion manager with Kotlin flow to avoid polling
QR-02	Sensor and OBD reflect in ≤ 2 seconds	Sensor sampling, OBD polling and Room Db operations execute on background dispatchers via coroutines
QR-03	Trip data not lost on connectivity loss	Room Db persistence on device for trip data
QR-04	Maintain testing coverage $\geq 80\%$	Individually testable layers and modules
QR-05	Availability $\geq 99\%$ uptime	Health check ping every minute (Azure App Services), and rate limiting on endpoints to avoid DOS attacks

Table 1: Mapping Quality Requirements to Architectural Decisions

3.10 NFR Traceability Matrix

ID	Quantified requirement	Tactic in SAS	Test / tool	Target / actual
QR-04	Maintain testing coverage	Individually testable layers and modules	codecov	$\geq 80\%$ / 86%
QR-05	Availability over 30 days before demo	Health check ping, and rate limiting	uptime	$\geq 99.5\%$ / 99.92%
QR-01	UI should be able to respond in a set amount of time	Kotlin flow	junit4	$\geq 200\text{ms}\%$ / $80\text{ms} \geq \%$
QR-03	UI alert latency	Kotlin flow	junit4	$\geq 3000\text{ms}\%$ / $4500\text{ms} \geq \%$
QR-05	Performance response under load for start trip and end trip	Background jobs and logging	K6	$\geq 200\text{ms}\%$ / $182\text{ms} \geq \%$

Table 2: NFR Traceability Matrix

4 Service Contract

Swagger-JS doc using the OpenAPI schema. Yaml comments for each endpoint are in their respective files.

Visualization: [Swagger Endpoints](#)

5 Technology Requirements

5.1 TR1 - Mobile Application Runtime:

Android device running Android 8.0 or higher

5.2 TR2 - Backend Runtime:

Node.js version 20 or higher

5.3 TR3 - Database:

PostgreSQL version 15 or higher

Prisma ORM

5.4 TR4 - Containerization:

Docker Desktop with Docker Compose v2 or higher

5.5 TR5 - Cloud Infrastructure:

Azure app service

Azure DB for PostgreSQL Flexible Server

Azure Container Registry

5.6 TR6 - Frontend Runtime:

Next.js 15

5.7 TR7 - Package Management:

Yarn 4 via Corepack

5.8 TR8 - Version Control:

Git version 2.x or higher

5.9 TR9 - Android Development Environment:

Android Studio

Retrofit (HTTP client for Android and Java)

5.10 TR10 - External APIs and Services:

Azure Maps Web SDK

Bluetooth OBD-||

carapi

Firebase Cloud Messaging

5.11 TR11 - Authentication:

JSON Web Tokens

5.12 TR12 - CI/CD Pipeline:

Github actions

5.13 Deployment Diagram

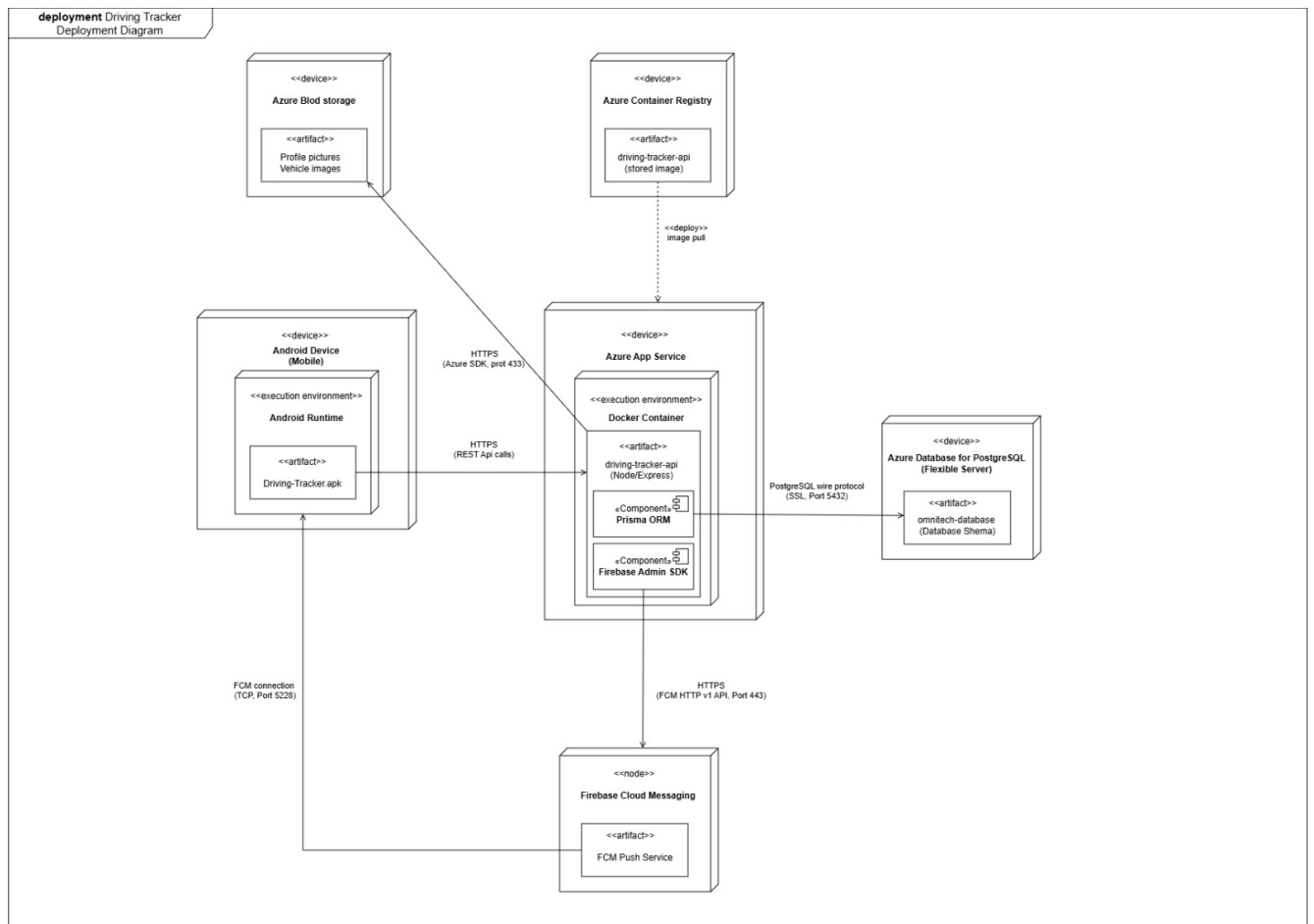


Figure 2: Deployment Diagram (including technologies)

5.14 CI pipeline flow diagram

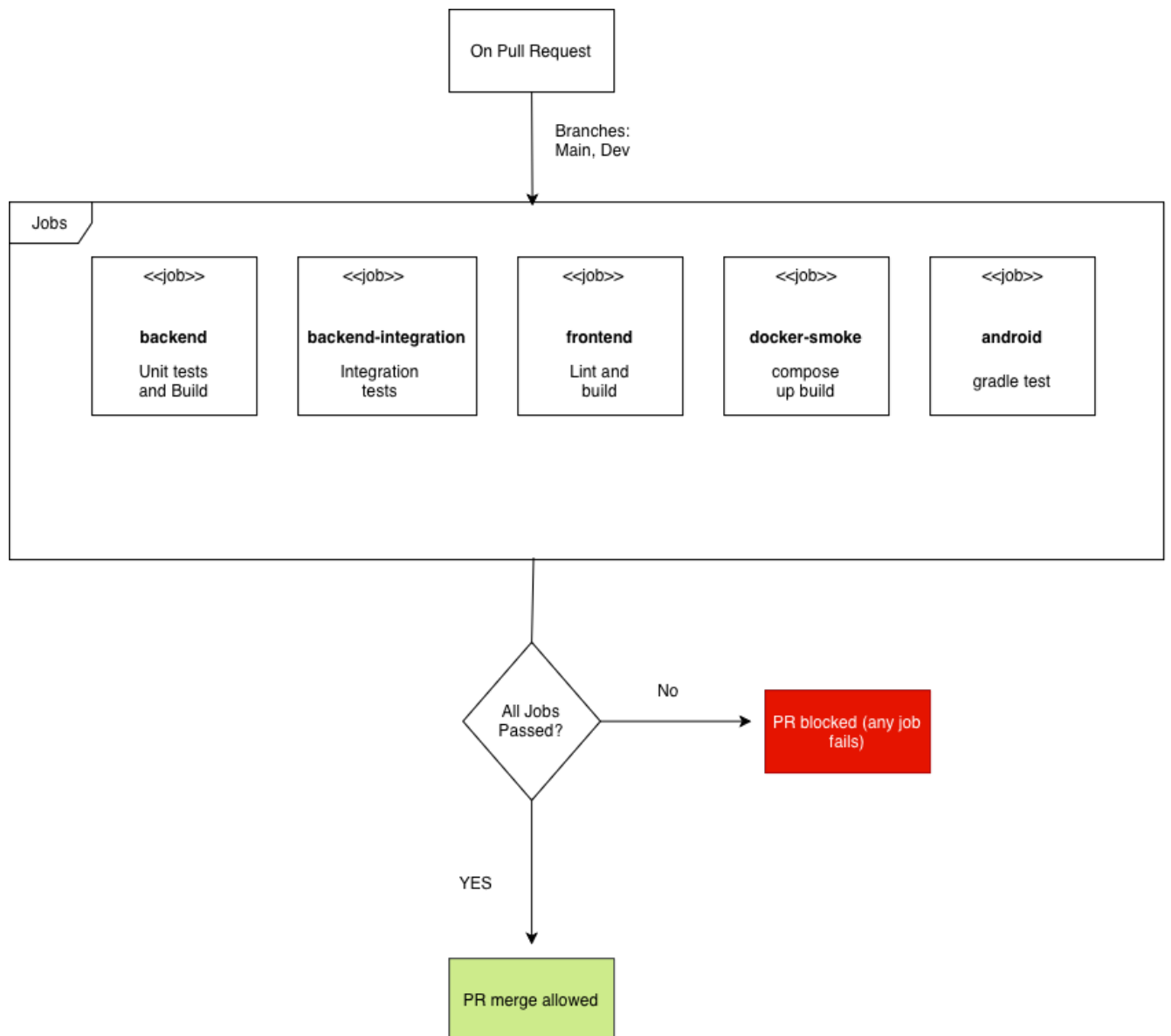


Figure 3: CI pipeline diagram

5.15 CD pipeline flow diagram

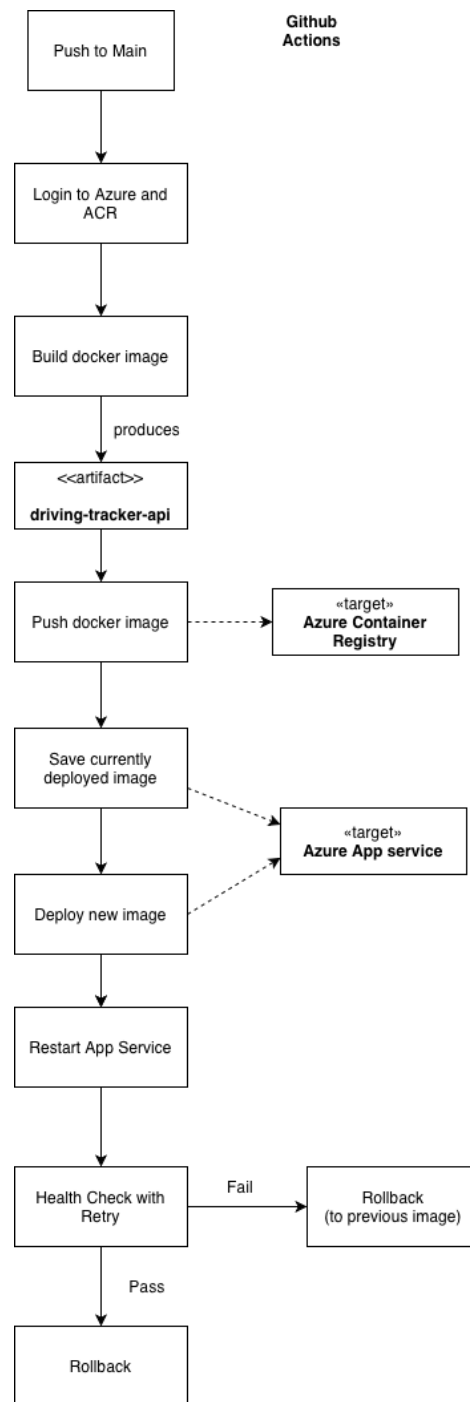


Figure 4: CD pipeline diagram